

2. Key Design Decisions

This section summarizes the decisions behind the design in `1-architecture-overview.md`. Where the full reasoning lives in `3-trade-offs.md` instead, this links to it rather than repeating it.

2.1 Service topology. Chosen: single deployable, internal CQRS split. Rejected: separate read/write service deployables. Why: fewer moving parts for MVP; split still available later (3.5).

2.2 Reliable publishing. Chosen: Transactional Outbox + Debezium CDC. Rejected: dual-write from the command handler; distributed transaction/2PC. Why: avoids the dual-write problem without 2PC complexity. Exception: Dispatcher (2.4) publishes directly — no DB write to protect.

2.3 Scheduler. Chosen: stateless, plan-agnostic daily tick. Rejected: a scheduler owning per-plan jobs (schedule/reschedule per plan). Why: editing/canceling a plan never touches scheduling infra.

2.4 Execution triggering at scale. Chosen: two-stage fan-out — Dispatcher scan (no writes) then partitioned Initiator group. Rejected: a single serial loop; a sharded consumer filtering the DB directly. Why: a serial loop doesn't scale at daily-aggregate volume and fails badly (3.2).

2.5 No-double-execution. Chosen: idempotency key (`plan_id`, `execution_month`), unique constraint. Rejected: relying on exactly-once delivery from Kafka or the Dispatcher's scan. Why: neither is guaranteed — both are at-least-once; the key makes every redelivery path safe by construction.

2.6 Concurrent plan edits. Chosen: Execution snapshots the Plan's investments once, at initiation. Rejected: reading the Plan's investments live/repeatedly during processing. Why: a concurrent edit can't corrupt an in-flight Execution — it only affects future ones. No locking needed.

2.7 Partial-failure handling. Chosen: Execution `Processed` tolerates partial Order rejection; per-instrument outcome lives at the Order level. Rejected: a distinct Execution-level `Failed/PartiallyProcessed` status. Why: 1 of 5 legs rejected isn't a failed execution, it's a partial success — avoids fuzzy failure-threshold semantics.

2.8 Trading outage / insufficient cash. Chosen: both surface identically as an async `OrderRejected`; no pre-check. Rejected: synchronously checking Cash balance or Trading/Orders health before issuing a command. Why: Cash allocation is the Cash Domain's job; Kafka durability already buffers commands through a Trading outage (3.4).

2.9 Order identity. Chosen: composite key (`execution_id`, `asset_id`). Rejected: a separate generated `order_id`. Why: an Execution has at most one Order per asset by construction, so the pair is already unique. See entity model, 1.9.

2.10 Plan Inactive recovery. Chosen: system- or user-triggered pause; only exit is a user edit; detection is reactive (via a failed execution attempt). Rejected: automatic recovery once a blocker resolves; proactive reconciliation against live asset data. Why: automatic recovery could silently resume a plan the user never re-confirmed; reactive is the cheaper MVP path (3.3).

2.11 Execution outcome notification. Chosen: publish enriched events on `investment-plan-data`; don't own delivery. Rejected: building/owning notification delivery ourselves; a dedicated notifications topic. Why: channel/timing/delivery isn't our domain's concern; matches existing subscription convention (3.4).